

02 | Component：搭建静态页面的正确思路是什么？

2022-3-30 蒋宏伟



你好，我是蒋宏伟。

上一讲我们说到，React/React Native 开启了“基于组件”构建应用的潮流。

在工作中，特别是业务类的开发需求，绝大多数都是写页面。写页面分为两步，第一步是搭建静态页面，第二步是给静态页面添加交互让它动起来。这第一步至关重要，它决定了 UI 设计稿要拆分成哪些组件，这些组件又是如何组织起来的，这些都会影响程序的可扩展性和可维护性，甚至还有团队的合作方法。

我们这一讲的目的，就是让你有一个正确的基于组件搭建静态页面的思路，不让第一步走偏。要知道，如果后面再去纠正，要花费的成本就大了去了。

组件：可组合、可复用的“拆稿”方式

在开始使用组件这种方式构建静态页面之前，请你先思考一个问题，为什么 React/React Native 选择了基于组件的架构方式呢？

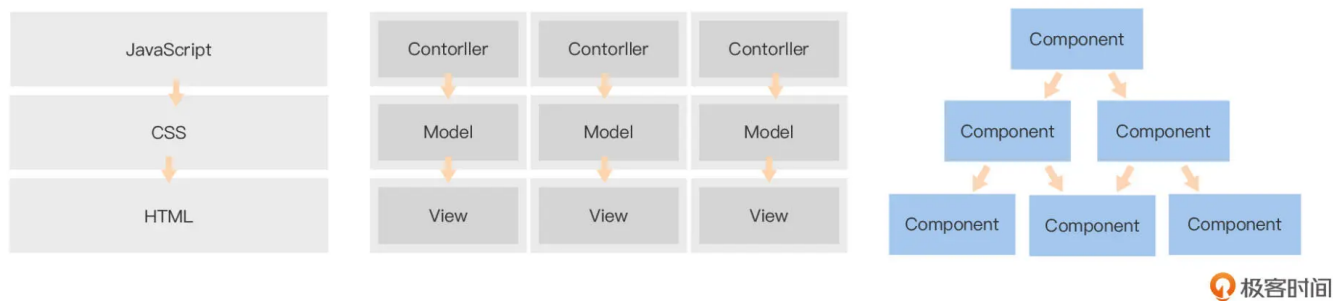
理论上，除了组件这种方式外，常见的构建应用方式还有：类似 HTML/CSS/JavaScript 这种的分层架构、基于 MVC 的分层架构。那为什么 React/React Native 没有选择这两种架构方式呢？

这是因为，**基于组件的架构模式，或许是现在重展示、重交互应用的最好选择。**

记得我 2015 年刚入门的时候，还有一种岗位叫做网页重构工程师，我还面过这种岗位。那个时候，架构模式就是把 UI 设计稿拆成 3 层：HTML、CSS、JavaScript。网页重构工程师负责 HTML、CSS 部分，前端工程师负责 JavaScript 部分。但是后来我发现网页重构工程师这种岗位越来越少了，也庆幸自己没有上错车。

现在，相信你也看到了，把 UI 设计稿拆成完全独立的 HTML/CSS/JavaScript三个部分的这种架构已经不是主流了；2010 年开源的、代表 MVC 架构模式的 AngularJS也被 Angular（v2 及更高版本）这种基于组件的架构模式所代替了；现在 iOS、Android 应用也有很多是基于组件开发的。

为什么会有这种现象呢？我先给你看一张架构对比图，你先可以体会一下它们之间的区别，找找原因：



现代应用都很复杂，而且非常重交互、重展示。如果 React Native 选择的是类似 HTML/CSS/JavaScript 的模板、样式、逻辑分离的分层架构，那可想而知，我们的三层代码都会非常臃肿。

如果 React Native 选择的是 MVC 架构，把逻辑控制、数据模型和视图进行分层，对程序横向分层纵向打通，这样代码颗粒度是会变小。但在重交互的前提下，层和层之间、列和列之间的数据流向却更复杂了。流动的方向不止是 MVC 架构图中画“3+3”的 6 个方向，而是层和层之间的“332”个方向，列和列之间的“332”个方向，非常复杂。

React/React Native 选择的是基于组件的架构模式，它有三个好处：

第一，组件是内聚的，组件内既有逻辑，又有状态，还有视图，一个组件可以独立完成一件事情，这也使得 UI 模块复用变得简单；

第二，组件之间是可以组合的，一个页面可以拆分成若干个大组件，大组件也可以拆分成小组件，当某个组件变大变臃肿时也可以进一步地拆分；

第三，组件和组件之间的数据流向永远是确定，永远是从上往下流动的，简单明了。

组件可组合、可复用的特性，和组件之间单向数据流的模式，在现代应用重交互重展示的情况下，显然更吃香，这也是 React/React Native 选择基于组件来构建应用的原因。

单一责任原则

现在我们回到第一步，基于组件搭建静态页面。

我们直接来看一个具体的例子。这里我放了一个简易商品列表页的 UI 设计稿，你可以先停下来思考一下，想一想你会把它拆成那些组件？你这么拆的原因又是什么？

名称	价格
水果	
PingGuo	¥ 1
HuoLongGuo	¥ 1
BaiXiangGuo	¥ 2
蔬菜	
BoCai	¥ 2
NanGua	¥ 4
WanDou	¥ 1

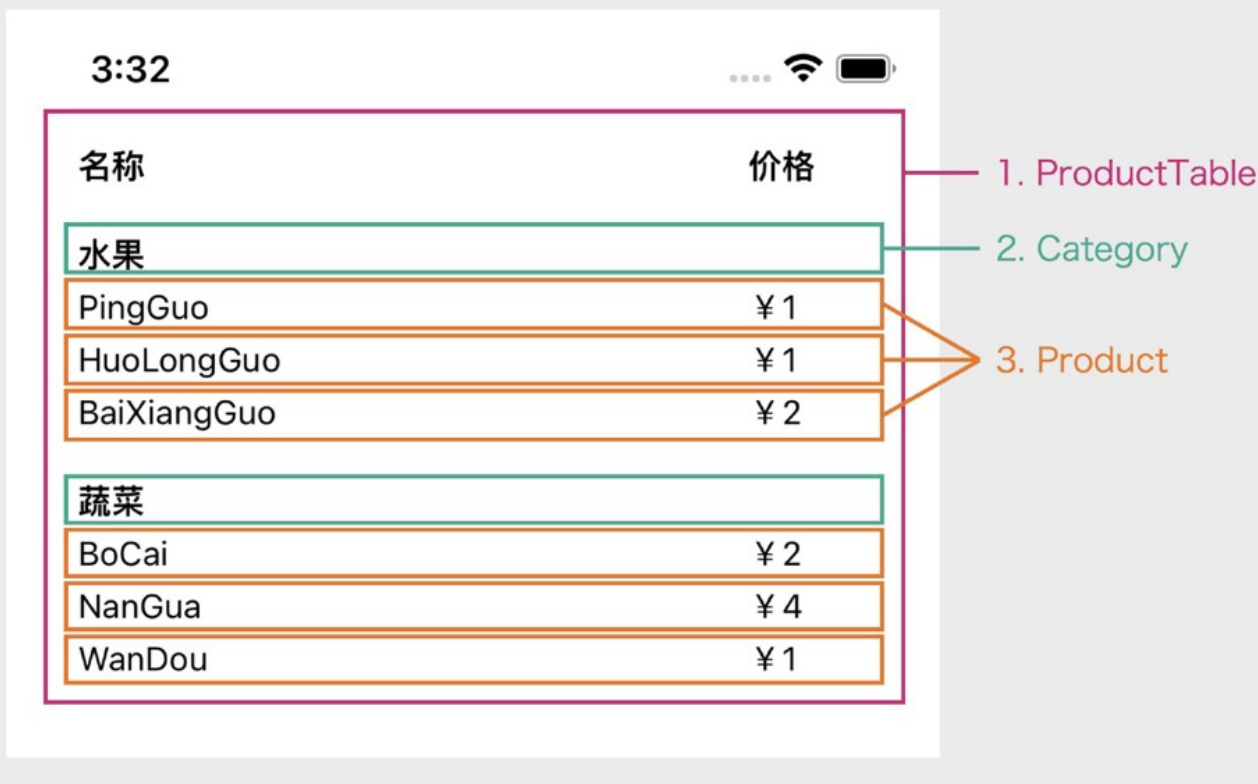
我们直接来揭晓答案，拆组件要准守一个原则，**单一责任原则**。

这也是 React 官方倡导的原则，这个原则的意思是**每个组件都应该只有一个单一的功能，并且这个组件和其他组件没有相互依赖**。当然，完全没有相互依赖是不可能的，但这种思路具有很高的指导价值，一个组件的依赖越少，设计得越好。

给你举个例子，一个组件你引用的依赖越多，这些依赖就像陌生的英语单词，你得去其他文件中去查词典，才能知道这些依赖的意思。依赖越多，越难读懂，也越难维护。

因此，为了可读性、可维护性、可测试性，就要减少组件的外部依赖，这就是单一责任原则的指导价值。

这样说来，在拆分简易商品列表页的 UI 设计稿时，我们就要尽可能地拆的更细一些，保证每个组件的责任单一，因为涉及到 UI 稿建议你打开文稿查看一下，那我们拆分结果如图所示：



你可以看到，这个简易商品列表已经被拆分了 3 个组件，具体如下：

1. ProductTable（紫色）：它是商品列表组件，显示商品列表和表头；
2. Category（青色）：它是类别组件，显示一类商品的种类；
3. Product（黄色）：它是商品组件，显示某个具体的商品名称和价格。

宿主组件：生产基础视图的工厂

当你有了怎么把 UI 设计稿拆分成组件的思路后，接下来就要构建静态页面了。

要构建静态页面，就要有基础的视图材料。在 React Native 中那些最基础、不可再拆的视图材料，大都是由 React Native 框架提供的**宿主视图**。

比如，UI 设计稿中的水果名称：“苹果”、“火龙果”，价格：“¥ 1”、“¥ 2”，还有最顶部的搜索框，这些都是宿主视图。

而生产宿主视图的工厂，就是宿主组件（Host Components）。这些**宿主组件通常是 React Native 框架提供的组件**，它们和你用 JavaScript 自定义的组件不同，宿主组件是直接由 iOS/Android 原生平台实现的。

除了 React Native 框架提供的宿主组件外，一些社区库也提供了宿主组件，甚至你自己也可以创建宿主组件。

它们共同的特点是，这些宿主组件上层是 JavaScript 部分，底层是 Native 部分，这两部分是通过 React Native 框架联系起来的。也就是说，你调用宿主组件时，底层直接渲染的是 Native 视图。

那么，我们这个简易商品列表页的 UI 设计稿中，用到了那些宿主组件呢？其实有三种：

容器组件 View：顾名思义它就是一个容器，可以用来包裹其他的组件，类似于 Web 中用于嵌套的 div；

文字组件 Text：设计稿中的文字，比如水果名字“苹果”、“梨子”，价格“1元”、“3元”等等，这些类似于 Web 中装载文字的 span。

安全区域组件 SafeAreaView：它是最外层的容器组件，用于适配 iPhoneX 等的刘海儿屏。

宿主组件就是一个生产基础视图的工厂，你可以用 Text 组件实例化不同的文字视图。比如，我们可以实例化一个“苹果”文字，也可以再实例化另一个“火龙果”文字，代码如下：

```
import {Text} from 'react-native';

const element1 = <Text>苹果</Text> // JSX
const element2 = <Text>火龙果</Text> // JSX
```

你看啊，在这段用 JavaScript 书写的代码中，使用了**类似 HTML 的声明式语法，JSX**。我们先从 react-native 框架中引入了 Text 组件，然后通过 JSX 语法，用一对单闭合标签将 Text 组件进行实例化，生成 Text 元素 element1。当 element1 这个元素渲染到手机屏幕上，就是文字“苹果”了，element2 就是文字“火龙果”。

复合组件：纯 JavaScript 函数

现在，你已经有了构建静态页面的宿主组件了，接下来你需要用这些宿主组件，搭建你自己事先拆好的自定义组件了，包括：

ProductTable 商品列表组件

Category 类别组件

Product 商品组件

要创建自定义的宿主组件，你必须写 Native 代码。但上面 3 个自定义组件，**你可以直接用 JavaScript 创建，不用写 Native 代码，这类组件也叫复合组件（Composite Components）**。这些复合组件是基于宿主组件或其他复合组件搭建而成的。

现在我们来创建第一个自定义的复合组件：Product 商品组件，它的示例代码如下：

```
export default function Product({product = {name: '苹果', price: '1元'}}) {
  return (
    <View style={{flexDirection: 'row', marginTop: 5}}>
      <Text style={{flex: 1}}>{product.name}</Text>
      <Text style={{width: 50}}>{product.price}</Text>
    </View>
  );
}
```

这段代码，对于一些新手来说可能有点长，我分四步和你解释：

第一步，导出组件。还记得单一责任原则吗？一个组件的责任要单一，一个文件的责任也要单一。因此通常一个文件中只有一个组件，用`export default`就可以将它导出，让其他文件`import`引入使用。

第二步，定义函数。组件是一种特殊的函数。组件名字的首字母一定是大写的，示例中的`Product`是组件，因此它的`P`是大写的（当然，还有类组件，但用得会越来越来少，这里我们不探讨，你可以自己额外搜些资料）。

第三步，接收入参。组件能从其父组件中接参数，而且组件是函数，因此该参数就是函数的入参，通常命名为属性`props`。`props`是一个对象，因此也可以直接对它进行解构，直接获取对象中的值。

示例代码中用的就是用解构的方式来获取参数的，它直接获取了`product`参数，这里的`product`是数据因此`p`是小写的。

第四步，返回 JSX。组件的返回值就是 JSX，我们前面也提到过，它是用来描述 UI 页面的，JSX 最终生成的是视图元素、文字元素。这里我们初始化了一个<view/>元素，和两个<Text/>元素。

我们概括一下，自定义复合组件就是一个纯粹的 JavaScript 函数，谁调用它，谁就可以给它传入参数，同样它调用谁，它就可以给谁传入参数，而 JSX 闭合标签就是调用函数的语法糖。

静态页面的最终实现

现在你知道了 Product 商品组件如何定义，那么 Category 类别组件、ProductTable 商品列表组件对你来说，也就很容易了。

最后我们来看下，静态页的最终实现，完整代码有点长，我就不都贴出来了，你可以看看文末补充材料中的链接，现在我们只看下它整体长什么样子：

```
// index.js
AppRegistry.registerComponent('appName', () => App);
```

```
// App.js
const PRODUCTS = [
  {category: '水果', price: '¥1', name: 'PingGuo'},
];
```

```
export default function App() {
  return (
    <SafeAreaView style={{marginHorizontal: 30}}>
      <ProductTable products={PRODUCTS} />
    </SafeAreaView>
  );
}
```

```
// ProductTable.js
import Category from './Category';
import Product from './Product';
```

```
export default function ProductTable({products}){
  // ...
  <Category category={products[i].category}
  // ...
```

```
<Product product={products[i]}  
  // ...  
}  
  
// Category.js  
export default function Category({category}){}  
  
// ProductTable.js  
export default function Product({product}) {}
```

这里我定义了五个文件，每个文件中都最多有一个的组件。

index.js 文件：它是根文件，在该文件中registerComponent方法，会调用根组件 App，然后开始逐级调用，渲染应用；

App 组件：在 App 组件中，用于表示商品信息的数据变量 PRODUCTS，在被调用时会通过 ProductTable 组件的 products 属性传递下去；

ProductTable 组件：它被 App 组件调用后，它的调用入参就是 products。products 是一个数组，数组中的每一项就是 Product组件的入参product。每一项中的分类，就是 Category 组件的入参 category。还是一样，组件首字母是大写的，属性、入参的首字母是小写的；

Category 组件：它会被 ProductTable 组件调用两次，第一次调用接收的入参category 是“水果”，第二次是“蔬菜”；

Product 组件：它会被 ProductTable 组件调用 6 次，生成 6 个不同的商品元素，展示在手机屏幕上。

简而言之，组件间的数据是单向流动的，是逐层往下传递的。调用是从根组件开始的，根组件会调用其子组件，子组件会调用子子组件，以此类推。调用过程中，数据会被当做组件的属性，层层传递下去。

总结

前面我们说了，React/React Native 之所以选择基于组件的方式来构建应用，原因就在于组件更能够满足现代应用重交互重展示的特点。

搭建 React Native 静态页面的核心就是搭建组件。它的整体思路是，从上往下拆出组件，从下往上把拆出来的组件进行逐一实现和拼装。

在这一讲中，我们搭建的静态页是一个无交互的、轻展示的应用，但 React/React Native 也表现得很好。只要我们遵循单一责任原则，对 UI 设计稿进行拆分，我们就能设计出一个可扩展的、可维护的应用。

即使后续这个应用有了复杂的交互、有了复杂的展示形式，它也能很好地扩展。我们只需把那些复杂的组件，那些不再符合单一责任原则的组件，进行拆分就可以了。

最后，请你牢牢记住，宿主组件是最基础的材料，所有我们自定义的复合组件都基于宿主组件搭建出来的，而复合组件又能搭建出更上层的复合组件，这样一步一步，我们才能把静态页面搭建完成。

补充材料

学习 React 最好的地方就是 [React 官网](#)。我给的官网地址是新官方地址，目前还是 beta 版本，但不妨碍它是学习 React 最好的地方。这一讲中商品列表静态页的案例，也是参考的 React 新官网改编的；

这节课里完整的商品列表静态页代码，我放在了 [GitHub](#) 上；

关于 React 为什么选择基于组件的架构方式，而不是 MVC，在 2013 年的 [《我们为什么要构建 React?》](#) 这篇文章汇中，React 团队给出了答案。

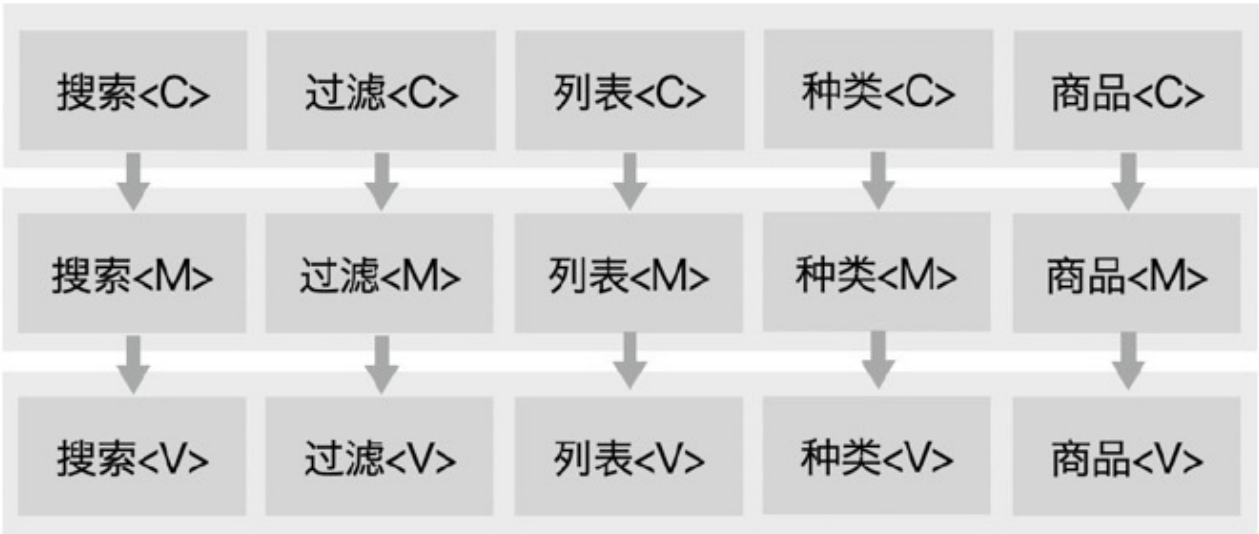
思考题

静态页面很难体现组件架构相对其他架构的优势。我再找了一个带交互的页面，这个页面可以搜索商品和过滤无库存的商品。请你思考一下，当我们按照搜索、过滤、列表、种类、商品五个维度，用 MVC 方式来架构页面时，它的数据流向是什么样的？它相对于组件架构的优点缺点又是什么？

搜索...

☐ 只显示有库存的商品

名称	价格
水果	
PingGuo	¥ 1
HuoLongGuo	¥ 1
BaiXiangGuo	¥ 2
蔬菜	
BoCai	¥ 2
NanGua	¥ 4
WanDou	¥ 1



上一篇 01 | 学习 React Native，你需要掌握哪些知识？

下一篇 03 | Style：关于样式你需要知道的三件事

精选留言 10